

Data_Project — תוכנית עבודה

פרויקט Zybo Z7-20 על כרטיס DDR Data Logger
מטרת הפרויקט: בניית מערכת FPGA מדורגת הכוללת RTL ב-Verilog, סימולציות, Vitis C, DDR, AXI DMA, AXI-Stream, CDC, FIFO, ובהמשך Python/CSV/Graph.

מצב נוכחי: שלבים 0-6 הושלמו. השלב הבא הוא Stage 7 - Custom IP Packaging + Vivado Block Design.

הושלם — Stage 0 — GitHub + Project Structure

סטטוס: הושלם

מטרה

להקים סביבת עבודה מסודרת ומקצועית לפרויקט, כך שכל הקוד, המסמכים, התמונות, הדוחות וקבצי הכלים יהיו מאורגנים במבנה שמתאים ל-GitHub ולתיק עבודות.

למה השלב חשוב

פרויקט FPGA מקצועי לא כולל רק קוד RTL. הוא צריך לכלול גם תיעוד, סימולציות, דוחות, תמונות, README ומבנה תיקיות ברור. זה מאפשר להציג את העבודה בצורה מסודרת למעסיקים ולהמשיך לפתח את הפרויקט בלי בלבול.

תוצר צפוי

Repository מסודר הכולל python, vitis, vivado, tb, rtl, images, docs, README ו-reports.

הושלם — Stage 1 — Data Protocol

סטטוס: הושלם

מטרה

להגדיר את פורמט חבילת הנתונים לפני כתיבת ה-RTL.

למה השלב חשוב

לפני שבונים מערכת שמעבירה נתונים, צריך לדעת בדיוק מה עובר במערכת. פרוטוקול הנתונים מגדיר את מבנה ה-Packet כך שכל בלוק בפרויקט ידע מה לצפות לקבל ומה להעביר הלאה.

תוצר צפוי

קובץ docs/data_protocol.md שמגדיר Expected Packet, Checksum, Data sequence, Length, Header, DATA_WIDTH ו-PASS / FAIL Criteria.

הושלם — Stage 2 — Data Generator + Testbench

סטטוס: הושלם

מטרה

לבנות את בלוק ה-RTL הראשון בפרויקט: Data Generator, ולבדוק אותו באמצעות Testbench וסימולציה ב-Vivado.

למה השלב חשוב

Data Generator משמש כמקור נתונים פנימי ומבוקר. במקום לחבר בשלב מוקדם מקור חיצוני כמו SPI, UART, מצלמה או חיישן, הבלוק מייצר סדרת נתונים ידועה מראש בתוך ה-FPGA.

תוצר צפוי

בלוק Data Generator עובד ומאומת בסימולציה, כולל קוד RTL, קובץ Testbench, תמונת waveform ודוח סימולציה.

הושלם — Stage 3 — Packet Builder FSM + Testbench

סטטוס: הושלם

מטרה

לבנות FSM שמקבלת נתונים מה-Data Generator ומרכיבה מהם Packet מלא לפי הפרוטוקול שהוגדר בשלב 1.

למה השלב חשוב

הבלוק מוסיף מבנה מסודר לנתונים. במקום להעביר רק מספרים גולמיים, הוא יוצר חבילה הכוללת Header, Length, Data ו-Checksum.

תוצר צפוי

rtl/packet_builder_fsm.v, קובץ Testbench וסימולציה שמוכיחה שה-Packet נבנה נכון.

הושלם — Stage 4 — Sync FIFO + Testbench

סטטוס: הושלם

מטרה

לבנות FIFO סינכרוני ולבדוק שמירה על סדר נתונים, כתיבה, קריאה, overflow, empty, full ו-underflow.

למה השלב חשוב

FIFO משמש כחוצץ בין בלוקים. הוא מאפשר לבלוק אחד לכתוב נתונים ולבלוק אחר לקרוא אותם בקצב מתאים, בלי לאבד את הסדר.

תוצר צפוי

rtl/sync_fifo.v, קובץ Testbench וסימולציה שמוכיחה שה-FIFO שומר על סדר הנתונים.

הושלם — Stage 5 — AXI-Stream Master + Testbench

סטטוס: הושלם

מטרה

לבנות בלוק שמוציא נתונים בממשק AXI-Stream בסיסי.

למה השלב חשוב

AXI-Stream הוא ממשק נפוץ להעברת נתונים רציפה בין בלוקים ב-FPGA, במיוחד כאשר עובדים עם AXI DMA.

תוצר צפוי

rtl/axi_stream_master.v, קובץ Testbench וסימולציה הכוללת בדיקת backpressure.

הושלם — Stage 6 — Full RTL Integration Simulation

סטטוס: הושלם

מטרה

לחבר את בלוקי ה-RTL המאומתים יחד בסימולציית אינטגרציה מלאה, בלי להסתיר אותם בתוך top סופי אחד.

למה השלב חשוב

השלב מוכיח שכל הבלוקים עובדים יחד: Data Generator, Packet Builder FSM, Sync FIFO ו-AXI-Stream Master. בנוסף, הוא שומר על הבלוקים נפרדים לקראת אריזת כל אחד מהם כ-Custom IP נפרד ב-Vivado.

תוצר צפוי

tb/tb_data_project_integration.v, סימולציית אינטגרציה מלאה, waveform ו-Console PASS שמראים שה-Packet המלא יוצא דרך AXI-Stream.

Stage 7 — Custom IP Packaging + Vivado Block Design

סטטוס: ממתין

מטרה

לארוז כל בלוק RTL כ-Custom IP נפרד ולחבר את הבלוקים בצורה חזותית ב-Vivado Block Design.

למה השלב חשוב

החלטנו להציג כל בלוק בנפרד כדי שהתצוגה תהיה ברורה לתיק עבודות ולראיונות. כך ניתן להסביר על כל בלוק בנפרד: Data Generator, Packet Builder FSM, FIFO ו-AXI-Stream Master.

תוצר צפוי

Custom IP נפרד לכל בלוק ו-Block Design חזותי שמכין את המערכת לחיבור AXI DMA ו-DDR.

Stage 8 — Vitis C Verification

סטטוס: ממתין

מטרה

לכתוב תוכנת C ב-Vitis שמפעילה את ה-DMA, קוראת נתונים מה-DDR ובודקת שה-Packet תקין.

למה השלב חשוב

ב-Zynq יש שילוב בין PL ל-PS. שלב זה בודק את החיבור בין החומרה שכתבנו לבין התוכנה שרצה על המעבד.

תוצר צפוי

תוכנית C שמפעילה DMA, קוראת DDR ומבצעת בדיקת Header / Length / Data / Checksum.

Stage 9 — Async FIFO + CDC + Testbench

סטטוס: ממתין

מטרה

להוסיף מעבר נתונים בין שני clock domains באמצעות Async FIFO.

למה השלב חשוב

CDC הוא נושא מרכזי בתעשייה. כאשר שני בלוקים עובדים עם שעונים שונים, אי אפשר להעביר נתונים ישירות בלי סנכרון מתאים.

תוצר צפוי

async_fifo.v וקובץ Testbench עם שני clocks וסימולציה שמוכיחה מעבר נתונים תקין.

Stage 10 — Full Integration with CDC + DMA + DDR

סטטוס: ממתין

מטרה

לשלב את ה-Async FIFO במערכת המלאה ולבדוק עבודה עם שני DMA, clock domains ו-DDR.

למה השלב חשוב

זה מחבר בין נושאי הליבה של הפרויקט: DMA, AXI, CDC, FIFO, FSM ו-DDR.

תוצר צפוי

מערכת מלאה עם שני domains וכתיבה תקינה ל-DDR.

Stage 11 — ILA + Vivado Reports

סטטוס: ממתין

מטרה

להוסיף ILA לדיבוג חומרה אמיתי ולשמור דוחות Vivado.

למה השלב חשוב

בסימולציה רואים מה אמור לקרות. בעזרת ILA אפשר לבדוק מה באמת קורה על הכרטיס. בנוסף, דוחות timing ו-utilization חשובים להוכחת איכות המימוש.

תוצר צפוי

screenshots של ILA, timing report, utilization report ותיעוד.

Stage 12 — Python CSV / Graph

סטטוס: ממתין

מטרה

לייצא את הנתונים לקובץ CSV וליצור גרף ב-Python.

למה השלב חשוב

זה מוסיף שכבת הצגה וניתוח נוחה, ומראה יכולת לשלב בין חומרה, תוכנה וכלי ניתוח.

תוצר צפוי

CSV, קוד Python, גרף ותיעוד תוצאה.

Stage 13 — UART or SPI Extension

סטטוס: ממתין

מטרה

להחליף את ה-Data Generator במקור נתונים אמיתי כמו UART או SPI.

למה השלב חשוב

זה הופך את המערכת ממקור נתונים פנימי למערכת שמסוגלת לקבל נתונים מבחוץ.

תוצר צפוי

בלוק תקשורת, בדיקה, וקבלת נתונים אמיתיים במערכת.

Stage 14 — Portfolio Polish

סטטוס: ממתין

מטרה

ללטש את הפרויקט כך שיהיה מוכן להצגה למעסיקים.

למה השלב חשוב

תיק עבודות מקצועי צריך להציג לא רק קוד, אלא גם הסברים, תמונות, waveforms, דוחות, מסקנות ו-Future Work.

תוצר צפוי

GitHub מסודר, README משופר, תמונות, דוחות, הסברים ולקחים מהפרויקט.
